Vulnlog Specification

Table of Contents

1. Overview . . . 2
   1.1. Scope . . . 2
   1.2. Design Principles . . . 3
   1.3. File Name Convention . . . 3
   1.4. Schema Reference . . . 3
   1.5. Multi-Branch Workflow . . . 3
2. File Structure . . . 4
   2.1. Ordering Rules . . . 4
   2.2. Unique Identifiers . . . 4
3. Sections . . . 5
   3.1. `schemaVersion` . . . 5
   3.2. `project` . . . 5
   3.3. `tags` . . . 5
   3.4. `releases` . . . 5
      3.4.1. PURL Entry . . . 6
   3.5. `vulnerabilities` . . . 6
      3.5.1. Resolution Entry . . . 7
      3.5.2. Report Entry . . . 8
      3.5.3. Suppress Entry . . . 8
4. Enumerations . . . 8
   4.1. Verdict Values . . . 8
   4.2. Severity Values . . . 9
   4.3. Justification Values (OpenVEX) . . . 9
   4.4. Reporter Type Values . . . 10
5. Defaults and Implicit Behavior . . . 10
6. Suppression Behavior . . . 11
   6.1. Suppression Rules . . . 11
   6.2. Suppression Identifier Resolution . . . 11
7. VEX Generation . . . 11
   7.1. Field Mapping . . . 11
   7.2. VEX Status Mapping . . . 12
   7.3. Vulnerability ID URL Mapping . . . 13
   7.4. Product Scoping via Tags . . . 14
   7.5. VEX Document Versioning . . . 14
8. HTML Report Generation . . . 14
   8.1. Report Filters . . . 15

8.2. Detail Pages (Future) . . . 15
9. Validation Rules . . . 15
10. CLI Design . . . 16
10.1. Usage . . . 16
10.2. Global Flags . . . 16
10.3. Filtering Flags . . . 16
10.4. Commands . . . 17
10.4.1. `vulnlog init` . . . 17
10.4.2. `vulnlog validate` . . . 17
10.4.3. `vulnlog lint` . . . 17
10.4.4. `vulnlog stats` . . . 18
10.4.5. `vulnlog report` . . . 18
10.4.6. `vulnlog suppress` . . . 19
10.4.7. `vulnlog vex` . . . 19
10.4.8. `vulnlog add` . . . 19
10.4.9. `vulnlog copy` . . . 19
10.4.10. `vulnlog sarif` (Future) . . . 20
10.5. Exit Codes . . . 20
11. Future Directions . . . 21
11.1. Multi-File Imports . . . 21
11.2. SARIF Export . . . 21
11.3. Per-Vulnerability HTML Detail Pages . . . 21
12. Examples . . . 21
12.1. Quick Start . . . 21
12.2. Full Example . . . 22

1. Overview

Vulnlog is a YAML-based format for tracking, analyzing, and managing software vulnerabilities reported by one or multiple Software Composition Analysis (SCA) scanners. Each Vulnlog file is scoped to a single release branch. The lifecycle of the file follows the lifecycle of the release branch — when a release branch reaches end of life (EoL), its Vulnlog file becomes a natural historical archive.

A Vulnlog file serves as a human-authored decision ledger. From this file, the Vulnlog CLI generates machine-readable outputs: SCA scanner suppression files, HTML reports, and OpenVEX statements.

1.1. Scope

Vulnlog is focused on security vulnerabilities reported by SCA scanners. License compliance findings, configuration issues, and other informational findings are out of scope for this version of the specification.

1.2. Design Principles

Compact and readable

The format prioritizes ease of writing and reading over exhaustive detail. The common case — reporting a finding and recording a verdict — should require minimal boilerplate.

IDE-supported

A JSON Schema published to SchemaStore provides validation, autocompletion, and inline documentation in editors.

Git-friendly

The file is designed for version control. Changes are tracked via Git history rather than in-file audit trails.

Data-oriented

Entries are plain data records with no behavior. The CLI applies transformations and business logic.

1.3. File Name Convention

Vulnlog files use the naming convention `<project-name>.vl.[yaml|yml]`. The `.vl.` infix identifies Vulnlog files and enables automatic JSON Schema association in IDEs via glob pattern matching. Simple projects may use `vulnlog.[yaml|yml]`.

The file location within the repository is not prescribed. The CLI requires an explicit file path as a positional argument; it does not search for files automatically.

1.4. Schema Reference

To enable IDE validation and autocompletion, add a schema reference comment as the first line of the Vulnlog file:

```
# $schema: .vulnlog/vulnlog-schema.json
```

The path can point to a local bundled schema file or to the published SchemaStore URL.

1.5. Multi-Branch Workflow

The recommended approach for teams managing multiple release branches is to maintain multiple Vulnlog files on the main branch, one per release branch:

```
demo-project-8.1.vl.yaml    # Release branch 8.1.x
demo-project-8.2.vl.yaml    # Release branch 8.2.x
demo-project-8.3.vl.yaml    # Release branch 8.3.x
```

Each file is self-contained. The CLI operates on one file at a time. This approach avoids cherry-picking vulnerability analysis to older Git branches. The Vulnlog file is archived together with the release branch once the version is EoL.

It remains possible to maintain a Vulnlog file directly on each release branch. Both approaches can coexist.

See `vulnlog copy` for CLI support to propagate entries across multiple files.

2. File Structure

```
# $schema: .vulnlog/vulnlog-schema.json
schemaVersion: "1"

project:
  # Project metadata

tags:
  # Tag definitions (optional)

releases:
  # Release definitions (chronological order, oldest first)

vulnerabilities:
  # Vulnerability entries (newest first)
```

Required top-level sections: `schemaVersion`, `project`, `releases`, `vulnerabilities`. Optional top-level sections: `tags`.

`vulnerabilities` may be an empty array.

2.1. Ordering Rules

Releases

Must be in chronological order (oldest first). This ordering has functional significance — the CLI uses array position to resolve `--release` range filtering. A release at position N includes all releases at positions 0 through N.

Vulnerabilities

Should be in reverse chronological order (newest first). The most recent findings appear at the top of the file for immediate visibility. This is a readability convention enforced by the linter, with no functional impact on CLI output.

2.2. Unique Identifiers

All vulnerability identifiers must be unique within a Vulnlog file. No two vulnerability entries may share the same `id`. An `id` of one entry must not appear in the `aliases` of another entry, and vice

versa. This ensures unambiguous lookup by any identifier.

3. Sections

3.1. `schemaVersion`

Field	Type	Required	Description
`schemaVersion`	String	Yes	Schema version of the Vulnlog file. For this specification it is `"1"`.

3.2. `project`

Project-level metadata. Used in report generation and as author information in VEX documents.

Field	Type	Required	Description
`organization`	String	Yes	Name of the organization or vendor.
`name`	String	Yes	Name of the software project.
`author`	String	Yes	Name of the responsible security team or author. Used as the author in VEX documents.
`contact`	String (email)	No	Contact email for the security team. Used as the author contact in VEX documents. This can be the contact information of your security.txt.

3.3. `tags`

Optional. Defines labels for categorizing and scoping vulnerability entries. Tags provide a flexible, team-defined taxonomy that can represent customer-facing deliverables (`binary`, `container`, `sdk`), internal usage contexts (`build-dep`, `test-infra`), or any other categorization.

Field	Type	Required	Description
`id`	String	Yes	Unique identifier for the tag. Referenced by vulnerability entries and release purl entries.
`description`	String	No	Human-readable description of what this tag represents.

3.4. `releases`

Defines the releases tracked in this Vulnlog file. Releases must be listed in chronological order (oldest first). This ordering is used by the CLI to resolve `--release` filtering.

Field	Type	Required	Description
`id`	String	Yes	Unique release identifier (e.g., `8.1.1`, `2.0.0-SNAPSHOT`).
`published_at`	String (date, `YYYY-MM-DD`)	No	Publication date of the release. Absence indicates the release is not yet published.
`purls`	Array of PURL Entry	No	Versioned Package URLs identifying the release artifacts. Used as product identifiers in VEX documents.

3.4.1. PURL Entry

A release artifact identified by its Package URL, optionally tagged for scoping.

Field	Type	Required	Description
`purl`	String (Package URL)	Yes	The versioned Package URL for this artifact in this release.
`tags`	Array of String	No	Tags associated with this purl. Used to match vulnerability tags for scoped VEX generation. Must reference tags defined in the `tags` section.

When a vulnerability entry has tags that match a purl's tags, only the matching purls are included as products in the VEX statement. Purls without tags are included in VEX statements regardless of vulnerability tags.

3.5. `vulnerabilities`

Each entry represents a single vulnerability finding and the human verdict about it.

NOTE The verdict should reflect the most severe impact across all listed packages. If a package appears in multiple contexts, the analysis should focus on the highest-risk context.

Field	Type	Required	Description
`id`	String	Yes	Primary identifier. Preferably a CVE ID. May be a scanner-specific ID if no CVE exists. Must be unique across all entries and aliases.
`name`	String	No	Common name if one exists (e.g., "Log4Shell", "Heartbleed"). Displayed prominently in reports.
`aliases`	Array of String	No	Alternative identifiers (e.g., GHSA IDs, scanner-specific IDs). Must not overlap with any `id` or `aliases` in other entries.
`releases`	Array of String	Yes	Release identifiers affected. Must reference IDs from the `releases` section.

Field	Type	Required	Description
`description`	String	No	Brief human-readable description. Optional to allow quick stub entries during triage.
`packages`	Array of String (Package URL)	Yes	Package URLs of affected dependencies.
`reports`	Array of Report Entry	Yes	Scanner reports that identified this vulnerability.
`tags`	Array of String	No	Tags for categorizing and scoping. Must reference tags defined in the `tags` section. Used for CLI filtering and VEX product scoping.
`analysis`	String	No	Free-text analysis and rationale for the triage decision. Maps to `impact_statement` in VEX for `not_affected` and `action_statement` for `risk_acceptable`.
`analyzed_at`	String (date, `YYYY-MM-DD`)	No	Date the analysis was performed. Must not be before the earliest report date.
`comment`	String	No	Additional information not related to the analysis itself. May include evidence references, methodology notes, decision-making context, or reminders for future review. Not used in any output generation.
`severity`	String (enum)	Conditional	Severity assessment. Required when `verdict` is `affected` or `risk_acceptable`. See Severity Values.
`verdict`	String (enum)	No	The human triage decision. See Verdict Values. Absence indicates under investigation.
`justification`	String (enum)	Conditional	Required when `verdict` is `not_affected`. See Justification Values (OpenVEX).
`resolution`	Resolution Entry	No	Records that the underlying dependency was updated or the vulnerability was otherwise addressed. Independent of the verdict. See Resolution Entry.

3.5.1. Resolution Entry

Records that a dependency update or other corrective action was taken, independent of the verdict. A vulnerability may be `not_affected` (the verdict) yet still have a `resolution` (the dependency was updated defensively). A vulnerability may be `affected` with a `resolution` indicating it was fixed in a subsequent release.

Field	Type	Required	Description
`in`	String	Yes	Release in which the resolution was applied. Must reference a release ID from the `releases` section.

Field	Type	Required	Description
`at`	String (date, `YYYY-MM-DD`)	No	Date the resolution was applied.
`ref`	String (URI)	No	Reference to the issue or ticket tracking the resolution.
`note`	String	No	Brief description of how the vulnerability was resolved.

3.5.2. Report Entry

Field	Type	Required	Description
`reporter`	String (enum)	Yes	The scanner or source type. See Reporter Type Values.
`at`	String (date, `YYYY-MM-DD`)	No	Date the vulnerability was reported. When omitted, the CLI uses the current date during output generation.
`source`	String	No	Description of the report source. Optional for scanner reporters. Required by the CLI for reporter type `other`.
`vuln_id`	Array of String	No	Scanner-specific vulnerability identifiers. Used for scanner-specific suppression files.
`suppress`	Suppress Entry	No	Scanner-specific suppression configuration. When present without `expires_at`, the suppression is permanent.

3.5.3. Suppress Entry

Field	Type	Required	Description
`expires_at`	String (date, `YYYY-MM-DD`)	No	Expiration date. After this date, the entry is excluded from the generated suppression file. When omitted, the suppression is permanent.

4. Enumerations

4.1. Verdict Values

The term "verdict" is used deliberately to distinguish from automated scanner statuses — it reflects a deliberate human judgment about the impact of a vulnerability.

Value	Description	OpenVEX Mapping
`affected`	The vulnerability affects this release. Requires `severity`. When a `resolution` is present, maps to VEX `fixed`.	`affected` (without resolution) or `fixed` (with resolution)
`not_affected`	The vulnerability does not affect this release. Requires `justification`.	`not_affected`
`risk_acceptable`	The vulnerability affects this release but the risk has been assessed and accepted. Requires `severity`.	`affected` (with `action_statement` from `analysis`)
(absent)	Not yet triaged.	`under_investigation`

NOTE "Fixed" is not a verdict. Fixing a vulnerability is a resolution action, not a judgment about impact. Use `verdict: affected` with a `resolution` block to record that a vulnerability was real and subsequently fixed. Use `verdict: not_affected` with an optional `resolution` to record that a vulnerability was not relevant but the dependency was updated anyway.

4.2. Severity Values

Required when `verdict` is `affected` or `risk_acceptable`.

Value	Description
`critical`	Immediate action required. Exploitable with severe impact.
`high`	Action required. Significant impact if exploited.
`medium`	Action recommended. Moderate impact.
`low`	Low priority. Minimal impact or difficult to exploit.

4.3. Justification Values (OpenVEX)

Value	Description
`component_not_present`	The vulnerable component is not included in the deliverable.
`vulnerable_code_not_present`	The vulnerable code is present but the specific vulnerable function or code path is not included.
`vulnerable_code_not_in_execute_path`	The vulnerable code is present but cannot be reached during execution.
`vulnerable_code_cannot_be_controlled_by_adversary`	The vulnerable code is reachable but cannot be triggered by an attacker.
`inline_mitigations_already_exist`	The vulnerability is mitigated by existing controls (e.g., WAF, input validation).

4.4. Reporter Type Values

Value	Scanner	Default Suppression Path	Suppression Support
`trivy`	Trivy	`.trivyignore`	Yes
`snyk`	Snyk	`.snyk`	Yes
`dependency_check`	OWASP Dependency Check	`dependency-check-suppressions.xml`	Yes
`grype`	Grype	`.grype.yaml`	Yes
`rust_audit`	Cargo Audit (Rust)	`.cargo-audit.toml`	Yes
`semgrep`	Semgrep	—	No
`npm_audit`	npm audit	—	No
`github_advisory`	GitHub Advisory Database	—	No
`other`	Generic / non-scanner source	—	No

For reporters of type `other`, the `source` field on report entries is required by the CLI.

5. Defaults and Implicit Behavior

Omitted `tags` section

The file has no tag definitions. VEX generation includes all purls as products.

Omitted `verdict`

Treated as `under_investigation`. The CLI warns about entries without a verdict.

Omitted `reports[].at`

The CLI uses the current date and time during output generation.

Omitted `suppress.expires_at`

The suppression is permanent (never expires).

Omitted `analyzed_at`

No analysis date is recorded.

Omitted `vulnerabilities[].tags`

The vulnerability is not scoped. Included in all unfiltered outputs.

Omitted `releases[].purls[].tags`

The purl is included in VEX statements regardless of vulnerability tags.

6. Suppression Behavior

6.1. Suppression Rules

Entries where `verdict` is `not_affected` are always included in suppression files.

Entries where `verdict` is `affected` with a `resolution` are **not** included in suppression files. The vulnerable dependency version should no longer be present after the resolution. If the scanner still reports the finding, the resolution is incomplete.

For all other cases (`affected` without resolution, `risk_acceptable`, absent), suppression requires an explicit `suppress` block on the report entry. If the `suppress` block includes `expires_at`, the suppression is temporary. If `expires_at` is omitted, the suppression is permanent.

6.2. Suppression Identifier Resolution

1. `vuln_id` values from the report entry for the target reporter type, if present.
2. The vulnerability's `id` field.

`aliases` are not used for suppression.

7. VEX Generation

OpenVEX documents are generated via the CLI. Vulnlog tags + release purls map to OpenVEX products. Vulnlog packages map to OpenVEX subcomponents.

7.1. Field Mapping

Table 1. Minimal required OpenVEX fields mapped to Vulnlog fields.

OpenVEX Field	Vulnlog Source
`@context`	Static value: `https://openvex.dev/ns/v0.2.0`
`@id`	Generated IRI: <code><a href="https://vulnlog.dev/vex/<UUID>" class="bare">https://vulnlog.dev/vex/<UUID></a>;</code>
`author`	`project.author`. When `project.contact` is present: `"<author> (<contact>)"`.
`timestamp`	Document generation timestamp.
`version`	`1` for new documents. Previous version + 1 when updating an existing document (see VEX Document Versioning).
`statements[].vulnerability.name`	`vulnerabilities[].id`

OpenVEX Field	Vulnlog Source
`statements[].status`	Derived from `verdict` and `resolution` (see VEX Status Mapping)
`statements[].justification`	`vulnerabilities[].justification`
`statements[].impact_statement`	`vulnerabilities[].analysis` (when verdict is `not_affected`)

7.2. VEX Status Mapping

Verdict	Resolution	OpenVEX Status
`affected`	absent	`affected`
`affected`	present	`fixed`
`not_affected`	any	`not_affected`
`risk_acceptable`	any	`affected` (with `action_statement` from `analysis`)
(absent)	any	`under_investigation`

Table 2. Additional fields to enrich the Vulnlog-generated OpenVEX document.

OpenVEX Field	Vulnlog Source
`role`	Static value: `Document Creator`
`statements[].vulnerability.@id`	Vulnerability ID reference URL (see Vulnerability ID URL Mapping)
`statements[].vulnerability.description`	`vulnerabilities[].description`
`statements[].timestamp`	`vulnerabilities[].analyzed_at` when present, otherwise earliest `reports[].at`, otherwise document generation timestamp.
`statements[].products[].@id`	PURLs from the target release's `purls` entries, filtered by matching tags
`statements[].products[].subcomponents[].@id`	`vulnerabilities[].packages[]`
`statements[].action_statement`	`vulnerabilities[].analysis` (when verdict is `risk_acceptable`)

Minimal Vulnlog-generated OpenVEX example.

```
{
  "@context": "https://openvex.dev/ns/v0.2.0",
  "@id": "https://vulnlog.dev/vex/550e8400-e29b-41d4-a716-446655440000",
  "author": "Acme Corp Security Team",
  "timestamp": "2026-03-10T00:00:00.000Z",
  "version": 1,
  "statements": [
    {
```

```
      "vulnerability": { "name": "CVE-2026-26996" },
      "status": "not_affected",
      "justification": "vulnerable_code_cannot_be_controlled_by_adversary",
      "impact_statement": "The minimatch is only used to generate the static project
website. Input is always trusted."
    }
  ]
}
```

Enriched Vulnlog-generated OpenVEX example.

```
{
  "@context": "https://openvex.dev/ns/v0.2.0",
  "@id": "https://vulnlog.dev/vex/550e8400-e29b-41d4-a716-446655440000",
  "author": "Demo Security Team (security@demo.com)",
  "role": "Document Creator",
  "timestamp": "2026-03-10T00:00:00.000Z",
  "version": 1,
  "statements": [
    {
      "vulnerability": {
        "@id": "https://nvd.nist.gov/vuln/detail/CVE-2021-44228",
        "name": "CVE-2021-44228",
        "description": "RCE vulnerability in Log4J via JNDI lookup"
      },
      "timestamp": "2021-12-10T00:00:00.000Z",
      "products": [
        {
          "@id": "pkg:generic/demo-project@8.1.1",
          "subcomponents": [
            { "@id": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1" }
          ]
        }
      ],
      "status": "fixed"
    }
  ]
}
```

NOTE The second example shows `verdict: affected` with a `resolution` mapping to VEX status `fixed`. The Vulnlog verdict records the impact judgment; the VEX status reflects the current state.

7.3. Vulnerability ID URL Mapping

Table 3. Vulnerability ID type to reference URL mapping.

Vulnerability ID Type	URL Prefix
CVE	<code><a href="https://nvd.nist.gov/vuln/detail/<ID>" class="bare">https://nvd.nist.gov/vuln/detail/<ID></a>;</code>
SNYK	<code><a href="https://security.snyk.io/vuln/<ID>" class="bare">https://security.snyk.io/vuln/<ID></a>;</code>
RUSTSEC	<code><a href="https://rustsec.org/advisories/<ID>" class="bare">https://rustsec.org/advisories/<ID></a>;</code>
GHSA	<code><a href="https://github.com/advisories/<ID>" class="bare">https://github.com/advisories/<ID></a>;</code>

7.4. Product Scoping via Tags

When a vulnerability has tags, the VEX generator matches those tags against purl entries on the target release. Only purls with matching tags are included as products. Purls without tags are always included. Vulnerabilities without tags are associated with all purls.

7.5. VEX Document Versioning

The OpenVEX `version` field is an incrementing integer. Since the CLI is stateless, the version is determined by reading the existing output file:

1. The CLI reads the file at the output path (specified by `-o` or the default `vex.json`).
2. If the file exists and is a valid OpenVEX document, the CLI extracts the current `version`.
3. The new document is generated with `version` = previous version + 1.
4. If no existing file is found, the document starts at version `1`.
5. The CLI writes the new document only if the statements have changed compared to the previous document. If unchanged, the file is not written and the CLI reports "No changes detected."

8. HTML Report Generation

The primary output is a self-contained HTML file with a summary table. Each vulnerability entry is a single row.

`name` (if present) is displayed prominently. `aliases` shown as secondary identifiers. Tags are

displayed for filtering context. `comment` is not included in HTML reports (it is internal-only metadata).

8.1. Report Filters

The HTML report provides interactive filters that can be combined:

Release
: Filter by one or more release identifiers.

Verdict
: Filter by verdict (`affected`, `not_affected`, `risk_acceptable`, under investigation).

Tags
: Filter by one or more tags.

Severity
: Filter by severity level (`critical`, `high`, `medium`, `low`).

Reporter
: Filter by reporter type to compare scanner coverage.

Resolution status
: Filter by resolved (has `resolution`) or unresolved entries.

Text search
: Free-text search across `id`, `name`, `aliases`, `description`, and `vuln_id`.

8.2. Detail Pages (Future)

A future version will generate per-vulnerability detail pages alongside the summary:

```
vulnlog-report/
  index.html                    # Summary table with links
  CVE-2021-44228.html           # Detail page
  ...
```

9. Validation Rules

1. `releases` in vulnerability entries must reference IDs defined in `releases`.
2. `tags` in vulnerability entries must reference IDs defined in `tags` (when `tags` section exists).
3. `tags` in release purl entries must reference IDs defined in `tags`.
4. `resolution.in` must reference a release ID defined in `releases`.
5. `reporter` must be a valid reporter type enum value.

6. `justification` is required when `verdict` is `not_affected` and forbidden otherwise.
7. `severity` is required when `verdict` is `affected` or `risk_acceptable`.
8. `source` is required on report entries when `reporter` is `other`.
9. All vulnerability `id` values must be unique.
10. No `id` may appear in any other entry's `aliases`, and no two entries may share an alias.
11. `suppress.expires_at`, when present, must be a valid future or present date at CLI execution time.
12. `analyzed_at`, when present, must not be before the earliest `at` date across reports for that vulnerability.
13. Entries in `releases` must be in chronological order (oldest first).
14. All entries in `packages` must be valid Package URLs.

10. CLI Design

10.1. Usage

```
vulnlog <command> <file...> [flags]
```

Every command except `init` requires one or more Vulnlog files as positional arguments. Read-only commands (`validate`, `lint`, `stats`) accept multiple files. Output-generating commands (`report`, `suppress`, `vex`) operate on a single file. No automatic file detection.

10.2. Global Flags

Flag	Description
`-q, --quiet`	Suppress non-essential output.
`-v, --verbose`	Verbose output.
`--no-color`	Disable colored output.
`--format <f>`	Output format for structured output: `text` (default), `yaml`, `json`.
`--version`	Print version and exit.
`--help`	Print help.

10.3. Filtering Flags

Available on `stats`, `report`, `suppress`, and `vex` commands.

Flag	Description
`--release <id>`	Include vulnerabilities affecting all releases up to and including this release.
`--tags <id>`	Filter by tag (repeatable).

10.4. Commands

10.4.1. `vulnlog init`

Scaffolds a minimal Vulnlog file containing only the required sections (`schemaVersion`, `project`, `releases`, `vulnerabilities`).

```
vulnlog init [flags]
```

Flag	Description
`-o, --output <path>`	Output file path (default: `vulnlog.yaml`).
`--project <n>`	Project name. Prompted if not provided.
`--organization <n>`	Organization name. Prompted if not provided.
`--author <n>`	Author name. Prompted if not provided.

10.4.2. `vulnlog validate`

```
vulnlog validate <file...> [--strict]
```

Validates one or more Vulnlog files against the schema and all validation rules. `--strict` treats warnings as errors (e.g., missing verdict, missing description).

10.4.3. `vulnlog lint`

```
vulnlog lint <file...> [--fix]
```

Enforces:

Top-level section order

`schemaVersion`, `project`, `tags`, `releases`, `vulnerabilities`.

Vulnerability field order

`id`, `name`, `aliases`, `releases`, `description`, `packages`, `reports`, `tags`, `analysis`, `analyzed_at`, `comment`, `severity`, `verdict`, `justification`, `resolution`.

Release chronological order

Oldest first.

Vulnerability reverse chronological order

Newest first (based on the earliest `reports[].at` date, entries without dates sorted last).

10.4.4. `vulnlog stats`

```
vulnlog stats <file...> [filter flags]
```

Prints summary statistics for one or more Vulnlog files. When multiple files are provided, per-file statistics are shown followed by a total.

Default output is a compact terminal format. Use `--format yaml` or `--format json` for machine-readable output.

Example output (multiple files):

```
branch-8.1.vl.yaml (Demo Project · 8.1.x)
──────────────────────────────────────────────────────────────────────────────
Vulnerabilities: 12
  affected: 2 · not_affected: 5 · risk_acceptable: 1 · under investigation: 4
Resolved: 3 of 12
Suppressions: 6 active · 2 expired

branch-8.2.vl.yaml (Demo Project · 8.2.x)
──────────────────────────────────────────────────────────────────────────────
Vulnerabilities: 8
  affected: 1 · not_affected: 4 · risk_acceptable: 0 · under investigation: 3
Resolved: 2 of 8
Suppressions: 4 active · 0 expired

Total (2 files)
──────────────────────────────────────────────────────────────────────────────
Vulnerabilities: 20
  affected: 3 · not_affected: 9 · risk_acceptable: 1 · under investigation: 7
Resolved: 5 of 20
Suppressions: 10 active · 2 expired
```

10.4.5. `vulnlog report`

```
vulnlog report <file> [filter flags] [--format html] [-o <path>]
```

Default output: `vulnlog-report.html`.

10.4.6. `vulnlog suppress`

```
vulnlog suppress <file> [--reporter <type>] [filter flags] [-o <path>]
```

When `--reporter` is specified, generates a single suppression file. When omitted, generates for all reporter types found in the vulnerability reports.

Output path: `--output` flag or default for the reporter type. `--output` is only valid with `--reporter`.

10.4.7. `vulnlog vex`

```
vulnlog vex <file> [filter flags] [-o <path>]
```

Default output: `vex.json`.

The CLI reads the existing file at the output path to determine the VEX document version. If the file exists, the version is incremented. If the content has not changed, the file is not rewritten. See VEX Document Versioning.

10.4.8. `vulnlog add`

Adds a vulnerability stub entry to one or more Vulnlog files. Use this to create a fresh entry when no existing entry exists to copy from.

```
vulnlog add <file...> --vuln-id <id> --reporter <type> [flags]
```

Flag	Description
`--vuln-id <id>`	Vulnerability ID to add (required).
`--reporter <type>`	Reporter type for the report entry (required).
`--release <id>`	Release to associate. When omitted, adds to all releases in each file.

The command inserts a minimal stub entry at the top of the vulnerabilities array in each file. If an entry with the same `id` already exists in a file, the command skips that file and reports a warning.

Example:

```
vulnlog add branch-8.1.vl.yaml branch-8.2.vl.yaml \
  --vuln-id CVE-2026-001 --reporter trivy
```

10.4.9. `vulnlog copy`

Copies a complete vulnerability entry from a source file into one or more target files. Use this to propagate an existing analysis across multiple branch files.

```
vulnlog copy <source-file> <target-file...> --vuln-id <id> [flags]
```

Flag	Description
`--vuln-id <id>`	Vulnerability ID to copy (required). Must exist in the source file.

The first positional argument is the source file. All subsequent positional arguments are target files. The entry is copied with all fields intact (analysis, verdict, justification, resolution, etc.). The `releases` field is adjusted to reference the latest published release that exist in each target file.

If an entry with the same `id` already exists in a target file, the command skips that file and reports a warning.

Example:

```
# Copy the analysis from the main branch file to older branch files
vulnlog copy branch-8.3.vl.yaml branch-8.1.vl.yaml branch-8.2.vl.yaml \
  --vuln-id CVE-2026-001
```

10.4.10. `vulnlog sarif` (Future)

Generates a SARIF document from Vulnlog data.

```
vulnlog sarif <file> [filter flags] [-o <path>]
```

Default output: `vulnlog.sarif.json`.

10.5. Exit Codes

Code	Name	Description
`0`	Success	Command completed successfully.
`1`	General error	Unexpected error (I/O failure, unhandled exception).
`2`	Validation error	The Vulnlog file contains validation errors.
`3`	File not found	The specified Vulnlog file does not exist.
`4`	Lint violation	The file has lint violations (without `--fix`).
`5`	Missing required flag	A required flag was not provided.
`6`	Invalid flag value	A flag references an unknown entity or unsupported type.

11. Future Directions

11.1. Multi-File Imports

A future version will introduce an `imports` directive for splitting a Vulnlog file into static project data and dynamic vulnerability data.

11.2. SARIF Export

See `vulnlog sarif` (Future).

11.3. Per-Vulnerability HTML Detail Pages

See HTML Report Generation.

12. Examples

12.1. Quick Start

```
# $schema: ../generated/vulnlog-bundled.json
---
schemaVersion: "1"

project:
  organization: Acme Corp
  name: Acme Web App
  author: Acme Corp Security Team

releases:
  - id: 1.0.0
    published_at: 2026-01-15

vulnerabilities:

  - id: CVE-2026-1234
    releases: [ 1.0.0 ]
    description: Remote code execution in example-lib
    packages: [ "pkg:npm/example-lib@2.3.0" ]
    reports:
      - reporter: trivy
    analysis: >
      The vulnerable code path is not reachable in our application
      because we only use the safe subset of the API.
    verdict: not_affected
    justification: vulnerable_code_not_in_execute_path
```

12.2. Full Example

```yaml
# $schema: ../generated/vulnlog-bundled.json
---
schemaVersion: "1"

project:
  organization: Demo Project Inc
  name: Demo Project
  author: Demo Security Team
  contact: security@demo.com

tags:
  - id: binary
    description: Demo Project binary file
  - id: container
    description: Demo Project container image
  - id: sdk
    description: Demo Project SDK
  - id: build-dep
    description: Internal build dependency

releases:
  - id: 8.1.0
    published_at: 2021-01-15
    purls:
      - purl: "pkg:generic/demo-project@8.1.0"
        tags: [ binary ]
      - purl: "pkg:docker/demo-project@8.1.0"
        tags: [ container ]
  - id: 8.1.1
    published_at: 2021-02-11
    purls:
      - purl: "pkg:generic/demo-project@8.1.1"
        tags: [ binary ]
      - purl: "pkg:docker/demo-project@8.1.1"
        tags: [ container ]
  - id: 8.1.2

vulnerabilities:

  # Report from a non-scanner source (Oracle CPU)
  - id: CVE-2022-21449
    name: Psychic Signatures
    releases: [ 8.1.2 ]
    description: ECDSA signature verification bypass in Java SE
    packages: [ "pkg:maven/com.oracle/jdk@17.0.2" ]
    reports:
      - reporter: other
        at: 2022-04-19
```

```
        source: "Oracle Critical Patch Update April 2022"
    tags: [ binary, container ]
    analysis: The project uses Java 17 and ECDSA signatures for JWT token
verification. Confirmed exploitable.
    analyzed_at: 2022-04-20
    severity: critical
    verdict: affected
    resolution:
      in: 8.1.2
      at: 2022-04-22
      ref: "https://jira.example.com/browse/SEC-55"
      note: "Updated JDK from 17.0.2 to 17.0.3"

  # Verdict: risk_acceptable - vulnerability present but risk accepted
  - id: SNYK-2021-003
    releases: [ 8.1.2 ]
    description: DoS vulnerability in some-other-dependency
    packages: [ "pkg:maven/com.some.other/some-other-dependency@3.11.0" ]
    reports:
      - reporter: snyk
        at: 2022-04-28
    tags: [ binary, container ]
    analysis: >
      Demo project utilizes some-other-dependency but only in rare cases.
      Therefore the risk is considered low and a fix would potentially
      be breaking for customer setups.
    analyzed_at: 2022-04-29
    comment: >
      Risk accepted by project lead (decision documented in RISK-12).
      Revisit when some-other-dependency 4.0 is released.
    severity: low
    verdict: risk_acceptable

  # Verdict: not_affected - OS-level vulnerability in container base image
  - id: CVE-2024-5535
    releases: [ 8.1.1 ]
    description: Buffer overread in OpenSSL SSL_select_next_proto
    packages: [ "pkg:apk/alpine/openssl@3.1.4-r0" ]
    reports:
      - reporter: trivy
        at: 2024-06-28
    tags: [ container ]
    analysis: >
      Vulnerable OpenSSL is present in the container base image but the affected
      NPN/ALPN negotiation code path is never invoked by our application.
    analyzed_at: 2024-06-29
    verdict: not_affected
    justification: vulnerable_code_not_in_execute_path

  # Verdict: affected - resolved in a subsequent release (named vulnerability)
  - id: CVE-2021-44228
```

```
    name: Log4Shell
    aliases: [ GHSA-jfh8-c2jp-5v3q ]
    releases: [ 8.1.1 ]
    description: RCE vulnerability in Log4J via JNDI lookup
    packages: [ "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1" ]
    reports:
      - reporter: trivy
        at: 2021-12-10
    tags: [ binary ]
    analysis: >
      Demo project utilizes Log4J and a PoC showed that the project
      is vulnerable to this RCE vulnerability.
    analyzed_at: 2021-12-10
    comment: >
      PoC exploit confirmed by Jane (security team) on 2021-12-10.
      See internal report attached to SEC-42.
    severity: critical
    verdict: affected
    resolution:
      in: 8.1.2
      at: 2021-12-12
      ref: "https://jira.example.com/browse/SEC-42"
      note: "Updated Log4J from 2.14.1 to 2.17.1"

  # Verdict: not_affected - with defensive resolution
  - id: CVE-2021-002
    aliases: [ GHSA-xxxx-yyyy ]
    releases: [ 8.1.1 ]
    description: Deserialization vulnerability in some-dependency
    packages: [ "pkg:maven/example.org/some-dependency@1.1.2" ]
    reports:
      - reporter: trivy
        at: 2021-03-03
      - reporter: snyk
        at: 2021-03-03
        vuln_id: [ SNYK-2021-002 ]
    tags: [ binary, container ]
    analysis: >
      Demo project does not utilize some-dependency
      and therefore is not affected.
    analyzed_at: 2021-03-03
    verdict: not_affected
    justification: vulnerable_code_not_present
    resolution:
      in: 8.1.2
      note: "Removed unused dependency some-dependency"

  # Stub entry - quick triage, no description, no verdict yet
  - id: CVE-2021-007
    releases: [ 8.1.1 ]
    packages: [ "pkg:maven/com.example/some-lib@4.0.0" ]
```

```
    reports:
      - reporter: trivy
        at: 2021-03-10
    tags: [ binary ]

  # No verdict - under investigation, with temporary suppression
  - id: CVE-2020-099
    releases: [ 8.1.0 ]
    description: Path traversal vulnerability in random-dependency
    packages: [ "pkg:maven/org.random/random-core@1.33.7" ]
    reports:
      - reporter: trivy
        at: 2020-12-28
        suppress:
          expires_at: 2021-01-02
    tags: [ build-dep ]
    analysis: >
      Vulnerability in internal build dependency.
      No fixed library available at the moment. Re-check later.
    comment: >
      Checked upstream issue tracker - fix expected in random-core 1.33.8.
      Monitor https://github.com/random/random-core/issues/421
    severity: low
```